

# Como fazer o jogo Termo no CodePen com React

Guia completo — do zero ao código funcionando. Baseado na metodologia das aulas de Front End da UniCesumar.

1

configuração

## Abrir o CodePen

Acesse [codepen.io](https://codepen.io) no navegador. Clique em "Start Coding" ou "Create Pen" (precisa de conta gratuita). Isso abre o editor com três painéis: HTML, CSS e JS.

[codepen.io](https://codepen.io) → botão "Create Pen" no canto superior direito

2

configuração

## Configurar o preprocessor Babel (obrigatório para JSX)

Como visto nas aulas: para executar JSX no CodePen é necessário mudar o preprocessor para Babel. Clique na engrenagem  da aba JS → selecione "Babel" no campo JavaScript Preprocessor → salve.

Sem o Babel o React não consegue interpretar a sintaxe JSX e você verá erros no console.

3

configuração

## Adicionar o HTML base

No painel HTML do CodePen, adicione apenas uma div com id "root" — é onde o React vai renderizar o jogo, igual ao padrão ensinado nas aulas.

---

```
<div id="root"></div>
```

---

4

configuração

## Importar React no topo do JS

No painel JS, cole as importações do React via CDN (esm.sh), exatamente como nas aulas do Prof. Dacio. O useState é importado de React — é o hook que "lembra" o estado do jogo.

---

```
import React, { useState } from "https://esm.sh/react"; import ReactDOM from
```

---

```
"https://esm.sh/react-dom"; import { createRoot } from "https://esm.sh/react-dom/client";
```

---

5

código

## Criar o componente Letter (quadrado de cada letra)

Seguindo o padrão de componentização das aulas: cada célula da grade é um componente reutilizável que recebe props (letra e estado: correta, presente ou ausente).

```
function Letter({ letter, status }) { const colors = { correct: "#538d4e", present:
"#b59f3b", absent: "#3a3a3c", empty: "transparent" }; return ( <div style={{ width: 62,
height: 62, border: "2px solid #565758", display: "flex", alignItems: "center",
justifyContent: "center", fontSize: 32, fontWeight: "bold", color: status !== "empty" ?
"#fff" : "#fff", background: colors[status] || "transparent", textTransform:
"uppercase" }}> {letter} </div> ); }
```

6

código

### Criar o componente Row (linha de 5 letras)

Igual ao board-row das aulas — agrupa 5 componentes Letter em uma linha usando flexbox.

```
function Row({ guess, evaluation, wordLength }) { const cells =
Array(wordLength).fill(null); return ( <div style={{ display: "flex", gap: 5 }}>
{cells.map((_, i) => ( <Letter key={i} letter={guess ? guess[i] || "" : ""}
status={evaluation ? evaluation[i] : "empty"} /> ))} </div> ); }
```

7

código

### Criar a lógica de verificação de palavra

Função pura (fora dos componentes) que compara o chute com a palavra secreta e retorna o status de cada letra: correta (verde), presente (amarelo) ou ausente (cinza).

```
function evaluateGuess(guess, secret) { const result =
Array(secret.length).fill("absent"); const secretArr = secret.split(""); const guessArr =
```

```
guess.split("")); // primeiro passo: letras corretas guessArr.forEach((l, i) => { if (l
=== secretArr[i]) { result[i] = "correct"; secretArr[i] = null; guessArr[i] = null; } });

// segundo passo: letras presentes guessArr.forEach((l, i) => { if (!l) return; const idx
= secretArr.indexOf(l); if (idx !== -1) { result[i] = "present"; secretArr[idx] = null; }

}); return result; }
```

## 8

código

### Criar o componente principal Game

Aqui fica o state centralizado no componente pai — como ensinado nas aulas (lifting state up).

O useState guarda o chute atual, a lista de tentativas e os resultados.

```
const WORD = "REACT"; const MAX_ATTEMPTS = 6; const WORD_LENGTH = WORD.length; function
Game() { const [guesses, setGuesses] = useState([]); const [evals, setEvals] =
useState([]); const [current, setCurrent] = useState(""); const [gameOver, setGameOver] =
useState(false); const [message, setMessage] = useState(""); function handleKey(e) { if
(gameOver) return; const key = e.key.toUpperCase(); if (key === "ENTER") { if
(current.length !== WORD_LENGTH) { setMessage("Palavra muito curta!"); return; } const
evaluation = evaluateGuess(current, WORD); const newGuesses = [...guesses, current];
const newEvals = [...evals, evaluation]; setGuesses(newGuesses); setEvals(newEvals);
setCurrent(""); if (current === WORD) { setMessage("🎉 Parabéns! Você acertou!");
```

```
setGameOver(true); } else if (newGuesses.length >= MAX_ATTEMPTS) { setMessage("😬 Fim de  
jogo! A palavra era: " + WORD); setGameOver(true); } else { setMessage(""); } } else if  
(key === "BACKSPACE") { setCurrent(prev => prev.slice(0, -1)); } else if (/^[A-Z]  
$/.test(key) && current.length < WORD_LENGTH) { setCurrent(prev => prev + key); } }  
React.useEffect(() => { window.addEventListener("keydown", handleKey); return () =>  
window.removeEventListener("keydown", handleKey); }, [current, guesses, gameOver]); const  
rows = Array(MAX_ATTEMPTS).fill(null); return ( <div style={{ display: "flex",  
flexDirection: "column", alignItems: "center", background: "#121213", minHeight: "100vh",  
paddingTop: 40, fontFamily: "sans-serif", color: "#fff" }}> <h1 style={{ fontSize: 32,  
letterSpacing: 4, marginBottom: 20 }}> TERMO </h1> <div style={{ display: "flex",  
flexDirection: "column", gap: 5 }}> {rows.map((_, i) => ( <Row key={i} guess={i <  
guesses.length ? guesses[i] : i === guesses.length ? current : ""} evaluation={evals[i]  
|| null} wordLength={WORD_LENGTH} /> ))} </div> {message && ( <p style={{ marginTop: 20,  
fontSize: 16 }}>{message}</p> )} {gameOver && ( <button onClick={() => { setGuesses([]);  
setEvals([]); setCurrent(""); setGameOver(false); setMessage(""); }} style={{ marginTop:
```

---

```
16, padding: "10px 24px", cursor: "pointer", fontSize: 15 }} > Jogar novamente
```

---

```
</button> }} </div> ); }
```

---

**9**

**código**

### **Renderizar o app no root (final do arquivo JS)**

Igual ao padrão das aulas: criar o root e renderizar o componente principal nele.

---

```
const root = createRoot(document.getElementById("root")); root.render(<Game />);
```

---

**10**

**resultado**

### **Testar o jogo**

No painel de preview do CodePen o jogo deve aparecer. Clique no preview e digite as letras pelo teclado. A palavra secreta padrão é REACT. Para trocar, basta mudar o valor da constante WORD no início do código JS.

O CodePen pode precisar de um clique no preview antes de capturar os eventos de teclado. Se não responder, clique uma vez na área do jogo.